

Modul 2: Backpropagation dan Automatic Differentiation

Nama: Fiodora Alysa Juandi NIM: 123450051 Kelas: RB
Tanggal: 2026-09-22

Simpan berkas ini sebagai `M02_NIM.ipynb` sebelum mulai mengerjakan.

Petunjuk

1. Ganti seluruh penanda `TODO`. Jangan menghapus sel pemeriksaan.
2. Nilai Kasus 1 tidak boleh diubah; seluruh angka pada modul mengacu padanya.
3. Gunakan `float64` untuk semua perhitungan gradien.
4. Tuliskan turunan manual pada sel markdown, bukan hanya di kertas.
5. Notebook harus lolos *Restart Kernel and Run All* sebelum dikumpulkan.
6. Luaran: `M02_NIM.ipynb`, `M02_NIM.pdf`, dan `M02_NIM_metrics.csv`.

```
import platform
import random

import numpy as np
import pandas as pd
import torch
from torch import nn

NIM = '123450051' # contoh: '120450123'
SEED = int(str(123450051)[-4:]) if str(123450051).isdigit() else 42
# seed individual
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
torch.set_default_dtype(torch.float64)
pd.set_option('display.precision', 8)
print({'python': platform.python_version(), 'numpy': np.__version__,
      'torch': torch.__version__, 'device': str(DEVICE), 'seed':
      SEED})

{'python': '3.13.14', 'numpy': '2.5.3', 'torch': '2.14.0+cpu',
 'device': 'cpu', 'seed': 51}
```

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum dimulai.

1. **Gradien lokal vs gradien total pada satu simpul:** TODO
2. **Mengapa `backward()` hanya dapat dipanggil pada tensor skalar:** TODO
3. **Isi `.grad` bila `backward()` dipanggil dua kali tanpa `zero_grad()`:** TODO
4. **Mengapa turunan BCE-with-logits terhadap logit berbentuk $p - y$, bukan $-y/p$:** TODO

Graf komputasi Kasus 1. Tuliskan urutan simpul dari x sampai L , lalu tandai gradien lokal di setiap simpul:

TODO

B. Turunan manual - 20 poin

Kasus 1 memakai nilai tetap berikut:

$$x = \begin{bmatrix} 2 & -1 \end{bmatrix}, W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 1 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 0 & 0 \end{bmatrix}, W^{(2)} = \begin{bmatrix} 2 & -1 \end{bmatrix}, b^{(2)} = 0.5, y = 1$$

Tuliskan penurunan Anda **berurutan** di sini, satu baris satu langkah:

1. $\partial L / \partial z^{(2)} =$? TODO
2. $\partial L / \partial W^{(2)} =$? TODO
3. $\partial L / \partial b^{(2)} =$? TODO
4. $\partial L / \partial h =$? TODO
5. $\partial L / \partial z^{(1)} =$? TODO
6. $\partial L / \partial W^{(1)} =$? TODO
7. $\partial L / \partial b^{(1)} =$? TODO

Cantumkan pula shape setiap gradien: TODO

```
x = np.array([2.0, -1.0])
W1 = np.array([[0.5, -0.5], [1.0, 1.0]])
b1 = np.array([0.0, 0.0])
W2 = np.array([2.0, -1.0])
b2 = 0.5
y = 1.0

def forward(x, W1, b1, W2, b2, y):
    """1: kembalikan dict berisi z1, h, z2, p, dan loss."""

    z1 = W1 @ x + b1
    h = np.maximum(0, z1)

    z2 = W2 @ h + b2
```

```

p = 1 / (1 + np.exp(-z2))

loss = -(y * np.log(p) + (1 - y) * np.log(1 - p))

return {
    'z1': z1,
    'h': h,
    'z2': z2,
    'p': p,
    'loss': loss
}
raise NotImplementedError

nilai = forward(x, W1, b1, W2, b2, y)
print({k: np.round(v, 6) for k, v in nilai.items()})

# Pemeriksaan wajib: jangan diubah.
assert np.allclose(nilai['z1'], [1.5, 1.0]), 'z1 belum benar'
assert np.isclose(nilai['z2'], 2.5), 'logit belum benar'
assert np.isclose(nilai['loss'], 0.0788897, atol=1e-6), 'loss belum benar'
print('forward pass sesuai Kasus 1')

{'z1': array([1.5, 1. ]), 'h': array([1.5, 1. ]), 'z2':
np.float64(2.5), 'p': np.float64(0.924142), 'loss':
np.float64(0.07889)}
forward pass sesuai Kasus 1

def backward(x, W1, W2, y, nilai):
    """ 2: kembalikan dict gradien untuk 'W1', 'b1', 'W2', 'b2'.

    Urutan pengerjaan: dz2 -> (dW2, db2, dh) -> dz1 -> (dW1, db1).
    Ingat gradien lokal ReLU dan bentuk perkalian luar untuk dW1.
    """

    z1 = nilai['z1']
    h = nilai['h']
    p = nilai['p']

    # Gradien pada output
    dz2 = p - y

    # Gradien layer kedua
    dW2 = dz2 * h
    db2 = dz2

    # Gradien yang mengalir ke hidden layer
    dh = dz2 * W2

    # Gradien lokal ReLU
    dz1 = dh * (z1 > 0)

```

```

# Gradien layer pertama
dW1 = np.outer(dz1, x)
db1 = dz1

return {
    'W1': dW1,
    'b1': db1,
    'W2': dW2,
    'b2': db2
}
raise NotImplementedError

grad_manual = backward(x, W1, W2, y, nilai)
for nama, v in grad_manual.items():
    print(f'{nama:>3}: {np.round(v, 7)}')

# Pemeriksaan wajib: dua angka kunci dari modul.
assert np.allclose(grad_manual['W2'], [-0.1137873, -0.0758582],
    atol=1e-6)
assert grad_manual['W1'].shape == W1.shape, 'shape dW1 harus sama dengan W1'
print('gradien manual sesuai angka acuan')

W1: [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]]
b1: [-0.1517164  0.0758582]
W2: [-0.1137873 -0.0758582]
b2: -0.0758582
gradien manual sesuai angka acuan

```

C. Autograd - 20 poin

Bangun ulang Kasus 1 dengan tensor PyTorch, lalu bandingkan gradiennya dengan hasil bagian B.

```

tW1 = torch.tensor(W1, requires_grad=True)
tb1 = torch.tensor(b1, requires_grad=True)
tW2 = torch.tensor(W2, requires_grad=True)
tb2 = torch.tensor(b2, requires_grad=True)
tx, ty = torch.tensor(x), torch.tensor(y)
kriteria = nn.BCEWithLogitsLoss()

def forward_torch():
    """3: hitung loss dengan BCEWithLogitsLoss pada LOGIT."""
    z1 = tW1 @ tx + tb1
    h = torch.relu(z1)

    z2 = tW2 @ h + tb2

```

```

    loss = kriteria(z2, ty)

    return loss
    raise NotImplementedError

loss = forward_torch()
loss.backward()

for nama, t in [('W1', tw1), ('b1', tb1), ('W2', tw2), ('b2', tb2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_manual[nama]))
    print(f'{nama}: autograd = {np.round(t.grad.numpy(), 7)}    selisih
maks = {selisih:.2e}')
    assert selisih < 1e-10, f'gradien {nama} belum cocok dengan hasil
manual'
print('autograd cocok dengan backward manual')

W1: autograd = [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]]    selisih maks = 0.00e+00
b1: autograd = [-0.1517164  0.0758582]    selisih maks = 0.00e+00
W2: autograd = [-0.1137873 -0.0758582]    selisih maks = 0.00e+00
b2: autograd = -0.0758582    selisih maks = 0.00e+00
autograd cocok dengan backward manual

# TODO 4: panggil backward() sekali lagi TANPA menghapus gradien,
#          cetak tw2.grad, lalu hapus gradien dan hitung ulang.
#          Jelaskan hasilnya pada sel markdown di bawah.
raise NotImplementedError

```

```

-----
NotImplementedError                                Traceback (most recent call
last)
Cell In[8], line 4
      1 # TODO 4: panggil backward() sekali lagi TANPA menghapus
gradien,
      2 #          cetak tw2.grad, lalu hapus gradien dan hitung ulang.
      3 #          Jelaskan hasilnya pada sel markdown di bawah.
----> 4 raise NotImplementedError

NotImplementedError:

```

Penjelasan akumulasi gradien:

Ketika `backward()` dipanggil dua kali tanpa menghapus gradien sebelumnya, nilai gradien menjadi sekitar **2 kali lipat** dari gradien pertama. Hal ini terjadi karena PyTorch mengakumulasi gradien pada atribut `.grad`.

Oleh karena itu, `optimizer.zero_grad()` diperlukan pada training loop untuk menghapus gradien dari iterasi sebelumnya sebelum menghitung gradien baru.

D. Gradient checking - 20 poin

Bandingkan gradien analitik dengan selisih terpusat:

$$g_{\text{num}} = \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}, \text{rel err} = \frac{|g_{\text{analitik}} - g_{\text{num}}|}{|g_{\text{analitik}}| + |g_{\text{num}}| + 10^{-12}}$$

Ambang lulus: seluruh baris di bawah 10^{-5} .

```
EPS = 1e-5

def loss_dengan(param, i, delta):
    """ 5: salin parameter, geser satu komponen sebesar delta,
    kembalikan loss."""
    W1_baru = W1.copy()
    b1_baru = b1.copy()
    W2_baru = W2.copy()
    b2_baru = b2

    if param == 'W1':
        W1_baru[i] += delta
    elif param == 'b1':
        b1_baru[i] += delta
    elif param == 'W2':
        W2_baru[i] += delta
    elif param == 'b2':
        b2_baru += delta
    else:
        raise ValueError(f'Parameter {param} tidak dikenali')

    hasil = forward(
        x,
        W1_baru,
        b1_baru,
        W2_baru,
        b2_baru,
        y
    )

    return hasil['loss']

    raise NotImplementedError

def finite_difference(param, i):
    """6: kembalikan gradien numerik dengan selisih terpusat."""
    loss_plus = loss_dengan(param, i, EPS)
    loss_minus = loss_dengan(param, i, -EPS)

    return (loss_plus - loss_minus) / (2 * EPS)
```

```

raise NotImplementedError

indeks = ([('W1', (0, 0)), ('W1', (0, 1)), ('W1', (1, 0)), ('W1', (1, 1))])
        + [('b1', (0,)), ('b1', (1,))]
        + [('W2', (0,)), ('W2', (1,))]
        + [('b2', ())])

baris = []
for nama, i in indeks:
    manual = grad_manual[nama][i] if i != () else grad_manual[nama]
    auto = {'W1': tW1, 'b1': tb1, 'W2': tW2, 'b2': tb2}
    [nama].grad.numpy()
    auto = auto[i] if i != () else auto
    numerik = finite_difference(nama, i)
    rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
    baris.append({'parameter': nama, 'indeks': str(i), 'manual':
manual,
                'autograd': float(auto), 'numerik': numerik,
                'rel_err': rel})

tabel = pd.DataFrame(baris)
print(tabel.to_string(index=False))
print('\nrelative error maksimum:', tabel['rel_err'].max())
assert len(tabel) == 9, 'tabel harus memuat sembilan komponen
parameter'
assert tabel['rel_err'].max() < 1e-5, 'masih ada baris yang melampaui
ambang'

parameter indeks      manual      autograd      numerik      rel_err
W1 (0, 0) -0.30343272 -0.30343272 -0.30343272 1.00453007e-10
W1 (0, 1)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 0)  0.15171636  0.15171636  0.15171636 2.95622632e-11
W1 (1, 1) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
b1 (0,) -0.15171636 -0.15171636 -0.15171636 2.95622632e-11
b1 (1,)  0.07585818  0.07585818  0.07585818 5.73360674e-11
W2 (0,) -0.11378727 -0.11378727 -0.11378727 6.76755052e-11
W2 (1,) -0.07585818 -0.07585818 -0.07585818 5.73360674e-11
b2 () -0.07585818 -0.07585818 -0.07585818 5.73360674e-11

relative error maksimum: 1.004530066507852e-10

# 7: simpan tabel ke M02_NIM_metrics.csv (ganti NIM dengan NIM Anda).
tabel.insert(0, 'run_id', 'gradcheck')
tabel.insert(1, 'seed', SEED)
tabel.to_csv(f'M02_{123450051}_metrics.csv', index=False)
print('tersimpan')

tersimpan

```

Checkpoint menit ke-95. Tunjukkan tabel sembilan baris di atas kepada asisten sebelum melanjutkan ke bagian E.

E. Diagnosis training loop - 20 poin

Fungsi `train_rusak` di bawah berjalan **tanpa pesan galat**, tetapi memuat **empat** kesalahan. Kasus yang dipakai adalah XOR dengan protokol modul: FNN $2 \rightarrow 4 \rightarrow 1$, SGD `lr=0.1`, 400 epoch, satu batch penuh.

```
X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])
```

```
def train_rusak(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)
    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.ReLU(),
        nn.Linear(4, 1),
    )
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor)
        opt.step()
        loss.backward()
        riwayat.append(loss.item())
    return model, riwayat
```

```
model_rusak, riwayat_rusak = train_rusak()
print(f'loss awal : {riwayat_rusak[0]:.4f}')
print(f'loss akhir : {riwayat_rusak[-1]:.4f}')
with torch.no_grad():
    print('prediksi :', (torch.sigmoid(model_rusak(X_xor)) >
0.5).int().flatten().tolist())
    print('target :', y_xor.int().flatten().tolist())
```

```
-----
-----
RuntimeError                                Traceback (most recent call
last)
Cell In[19], line 23
    19     loss.backward()
    20     riwayat.append(loss.item())
    21     return model, riwayat
    22
--> 23 model_rusak, riwayat_rusak = train_rusak()
    24 print(f'loss awal : {riwayat_rusak[0]:.4f}')
```

```
25 print(f'loss akhir : {riwayat_rusak[-1]:.4f}')
26 with torch.no_grad():
```

Cell In[19], line 19, in train_rusak(epoch, lr)

```
15     for _ in range(epoch):
16         logits = model(X_xor)
17         loss = kriteria(torch.sigmoid(logits), y_xor)
18         opt.step()
--> 19         loss.backward()
20         riwayat.append(loss.item())
21     return model, riwayat
```

File ~\Modul-Praktikum-Deep-Learning\.venv\Lib\site-packages\torch_tensor.py:623, in Tensor.backward(self, gradient, retain_graph, create_graph, inputs)

```
613 if has_torch_function_unary(self):
614     return handle_torch_function(
615         Tensor.backward,
616         (self,),
617         (...), 621         inputs=inputs,
618         622     )
--> 623 torch.autograd.backward(
624     self, gradient, retain_graph, create_graph, inputs=inputs
625 )
```

File ~\Modul-Praktikum-Deep-Learning\.venv\Lib\site-packages\torch\autograd_init__.py:395, in backward(tensors, grad_tensors, retain_graph, create_graph, grad_variables, inputs)

```
390     retain_graph = create_graph
392 # The reason we repeat the same comment below is that
393 # some Python versions print out the first line of a multi-
line function
394 # calls in the traceback and some print out the last line
--> 395 _engine_run_backward(
396     tensors,
397     grad_tensors_,
398     retain_graph,
399     create_graph,
400     inputs_tuple,
401     allow_unreachable=True,
402     accumulate_grad=True,
403 )
```

File ~\Modul-Praktikum-Deep-Learning\.venv\Lib\site-packages\torch\autograd\graph.py:1072, in _engine_run_backward(t_outputs, *args, **kwargs)

```
1069 torch._C._stash_obj_in_tls("context",
contextvars.copy_context())
1071 try:
-> 1072     return Variable._execution_engine.run_backward( # Calls
```

```

into the C++ engine to run the backward pass
1073         t_outputs, *args, **kwargs
1074     ) # Calls into the C++ engine to run the backward pass
1075 finally:
1076     if attach_logging_hooks:

```

RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation: [torch.DoubleTensor [4, 1]], which is output 0 of AsStrided, is at version 2; expected version 1 instead. Hint: enable anomaly detection to find the operation that failed to compute its gradient, with `torch.autograd.set_detect_anomaly(True, check_nan=False)`.

No	Baris kode bermasalah	Mengapa keliru	Gejala yang terlihat
1	Tidak ada aktivasi di antara dua <code>Linear</code>	Model tetap linear dan tidak cocok untuk XOR	Prediksi XOR tidak tepat
2	<code>kriteria(torch.nn.Sigmoid(logits), y_xor)</code>	<code>BCEWithLogitsLoss</code> harus menerima logit langsung	Loss tidak optimal
3	<code>opt.step()</code> sebelum <code>loss.backward()</code>	Parameter di-update sebelum gradien dihitung	Training tidak berjalan benar
4	Tidak ada <code>opt.zero_grad()</code>	Gradien lama terus menumpuk	Gradien terakumulasi

```

def train_benar(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.ReLU(), # Perbaikan 1: tambahkan aktivasi
nonlinier
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        # Perbaikan 2: hapus gradien lama
        opt.zero_grad()

        logits = model(X_xor)

```

```

    # Perbaikan 3: BCEWithLogitsLoss menerima logit langsung
    loss = kriteria(logits, y_xor)

    # Perbaikan 4: backward dulu, baru optimizer.step()
    loss.backward()
    opt.step()

    riwayat.append(loss.item())

return model, riwayat

model_benar, riwayat_benar = train_benar()

print(f'loss awal   : {riwayat_benar[0]:.4f}')
print(f'loss akhir  : {riwayat_benar[-1]:.4f}')

with torch.no_grad():
    print(
        'prediksi   :',
        (torch.sigmoid(model_benar(X_xor)) > 0.5)
        .int()
        .flatten()
        .tolist()
    )
    print(
        'target      :',
        y_xor.int().flatten().tolist()
    )

loss awal   : 0.7079
loss akhir  : 0.5228
prediksi   : [1, 1, 1, 0]
target      : [0, 1, 1, 0]

def train_rusak(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.ReLU(),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)

```

```
# Masih sengaja salah: sigmoid diberikan sebelum BCEWithLogitsLoss
```

```
loss = kriteria(torch.sigmoid(logits), y_xor)
```

```
# Urutan dibuat tidak crash
```

```
loss.backward()
```

```
opt.step()
```

```
# Masih sengaja salah: tidak ada zero_grad()
```

```
riwayat.append(loss.item())
```

```
return model, riwayat
```

```
model_rusak, riwayat_rusak = train_rusak()
```

```
print("Loss awal :", riwayat_rusak[0])
```

```
print("Loss akhir :", riwayat_rusak[-1])
```

```
Loss awal : 0.7349948557721859
```

```
Loss akhir : 0.6931471805599454
```

```
print("Loss awal :", riwayat_rusak[0])
```

```
print("Loss akhir :", riwayat_rusak[-1])
```

```
riwayat_rusak
```

```
Loss awal : 0.7349948557721859
```

```
Loss akhir : 0.6931471805599454
```

```
[0.7349948557721859,  
0.734780856693479,  
0.7343548873291023,  
0.7337210643546044,  
0.7328857007429049,  
0.7318574443288743,  
0.7306474188433909,  
0.7292693330315431,  
0.7277395197996459,  
0.7260768681003303,  
0.7243026166498744,  
0.7224399910332648,  
0.7205136835175648,  
0.7185491957070167,  
0.7165720844988775,  
0.7146071675632878,  
0.7126777522769001,  
0.7108049498772702,  
0.7090071250889719,  
0.7072995133216724,  
0.7056940168142094,  
0.7041991719493801,
```

0.7028202654995228,
0.701559569212044,
0.7004166596563406,
0.699388792195306,
0.698471302407434,
0.6976580135251559,
0.6969416332847712,
0.696314127478674,
0.6957670604772976,
0.6952918953637219,
0.6948802484732506,
0.6945240953126182,
0.6942159271198245,
0.6939488596166069,
0.6937166976043583,
0.6935139607526655,
0.6933358770762277,
0.6931783511308783,
0.6930379139157811,
0.6929116609503086,
0.6927971841427427,
0.6926925020328372,
0.6926316832147095,
0.6925889946635178,
0.692546812776442,
0.6925046741565962,
0.6924622975551752,
0.692419525964663,
0.6923762797052634,
0.6923325192157694,
0.6922882161001,
0.6922433309520031,
0.6921977965357371,
0.6921515050025212,
0.6921042979428043,
0.6920559581920455,
0.6920062024073822,
0.6919546735025982,
0.6919009320592983,
0.6918444458131499,
0.6917845762334436,
0.6917205610556352,
0.6917699517986421,
0.6918647038022551,
0.6919517304278937,
0.6920317693955518,
0.6921054896986821,
0.6921734966159667,
0.6922363366395321,

0.6922945022561846,
0.692348436534983,
0.6923985374881805,
0.6924451621837249,
0.6924886305965212,
0.6925292291928594,
0.6925672142481045,
0.6926028149021648,
0.6926362359606316,
0.692667660451979,
0.6926972519529868,
0.6927251566957415,
0.6927515054702671,
0.6927764153371592,
0.6927999911646009,
0.6928223270039061,
0.6928435073173178,
0.6928636080712323,
0.6928812716827504,
0.692894194181308,
0.692906644505788,
0.6929186263183107,
0.6929301445661569,
0.6929412053731177,
0.6929518159284604,
0.6929619843754679,
0.6929717197011848,
0.6929810316287224,
0.6929899305132043,
0.6929984272422025,
0.6930065331413106,
0.6930142598853142,
0.6930216194152711,
0.6930286238616699,
0.6930352854737357,
0.6930416165548494,
0.6930476294039738,
0.6930533362629203,
0.6930587492692382,
0.6930638804144746,
0.693068741507527,
0.6930733495194797,
0.6930777125799017,
0.6930818376147136,
0.6930857358048285,
0.6930894179700399,
0.6930928945653685,
0.6930961756791656,
0.6930992710328425,

0.6931021899820938,
0.6931049415194772,
0.6931075342782167,
0.6931099765370957,
0.693112276226317,
0.6931144409342007,
0.6931164779146126,
0.6931183940950075,
0.6931201960849891,
0.6931218901852898,
0.6931234823970854,
0.6931249784315616,
0.6931263837196622,
0.6931277034219501,
0.6931289424385245,
0.6931301054189388,
0.6931311967720717,
0.6931322206759133,
0.6931331810872233,
0.6931340817510382,
0.6931349262099912,
0.6931357178134298,
0.693136459726307,
0.6931371549378326,
0.6931378062698698,
0.6931384163850718,
0.6931389877947463,
0.6931395228664452,
0.6931400238312778,
0.6931404927909443,
0.6931409317244912,
0.6931413424947908,
0.6931417268547471,
0.6931420864532325,
0.6931424228407609,
0.6931427374749022,
0.6931430317254448,
0.6931433068793129,
0.6931435641452461,
0.6931438046582485,
0.6931440294838159,
0.6931442396219467,
0.693144436010948,
0.6931446195310415,
0.6931447910077807,
0.6931449512152829,
0.6931451008792888,
0.6931452406800536,
0.6931453712550784,

0.6931454932016912,
0.6931456070794811,
0.6931457134125957,
0.693145812691907,
0.693145905377053,
0.6931459918983612,
0.6931460726586598,
0.6931461480349828,
0.693146218380174,
0.6931462840243976,
0.6931463452765562,
0.6931464024256255,
0.6931464557419084,
0.693146505478213,
0.693146551870959,
0.6931465951412159,
0.69314663549568,
0.6931466731275875,
0.6931467082175751,
0.6931467409344846,
0.6931467714361188,
0.69314679986995,
0.6931468263737841,
0.6931468510763834,
0.6931468740980493,
0.6931468955511699,
0.6931469155407314,
0.6931469341647983,
0.6931469515149622,
0.693146967676763,
0.6931469827300822,
0.693146996749512,
0.6931470098046999,
0.6931470219606729,
0.6931470332781383,
0.6931470438137675,
0.6931470536204599,
0.6931470627475915,
0.6931470712412451,
0.6931470791444276,
0.6931470864972731,
0.6931470933372305,
0.6931470996992426,
0.6931471056159102,
0.6931471111176478,
0.6931471162328275,
0.693147120987915,
0.6931471254075953,
0.6931471295148912,

0.6931471333312738,
0.6931471368767652,
0.6931471401700355,
0.6931471432284926,
0.6931471460683657,
0.6931471487047847,
0.6931471511518537,
0.6931471534227187,
0.6931471555296326,
0.6931471574840142,
0.6931471592965049,
0.6931471609770201,
0.6931471625347981,
0.6931471639784464,
0.6931471653159826,
0.6931471665548755,
0.6931471677020814,
0.6931471687640788,
0.6931471697469005,
0.6931471706561639,
0.6931471714970994,
0.6931471722745752,
0.6931471729931244,
0.693147173656965,
0.6931471742700228,
0.6931471748359512,
0.6931471753581494,
0.6931471758397796,
0.6931471762837833,
0.6931471766928969,
0.6931471770696648,
0.6931471774164532,
0.6931471777354616,
0.6931471780287357,
0.6931471782981758,
0.6931471785455487,
0.6931471787724959,
0.6931471789805426,
0.6931471791711051,
0.6931471793454997,
0.6931471795049481,
0.6931471796505846,
0.6931471797834623,
0.6931471799045587,
0.6931471800147806,
0.6931471801149698,
0.6931471802059062,
0.693147180288314,
0.6931471803628639,

[illegible]

[illegible]

```
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599454,  
0.6931471805599454,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599454,  
0.6931471805599454,  
0.6931471805599454,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599453,  
0.6931471805599454]
```

TODO 10: gabungkan catatan tahap perbaikan ke metrics.csv.

```
tahap = [  
    {  
        'tahap': 'train_rusak',  
        'loss_akhir': riwayat_rusak[-1],  
    },  
    {  
        'tahap': 'train_benar',  
        'loss_akhir': riwayat_benar[-1],  
    }  
]
```

```
df_tahap = pd.DataFrame(tahap)  
df_tahap.insert(0, 'seed', SEED)
```

```
df_tahap.to_csv(
    f'M02_{NIM}_metrics.csv',
    index=False
)

print(df_tahap.to_string(index=False))
```

```
seed      tahap  loss_akhir
51 train_rusak  0.69314718
51 train_benar  0.52281674
```

```
# Tandai asal data
tabel_metrics = tabel.copy()
tabel_metrics.insert(0, 'bagian', 'gradient_check')

df_tahap_metrics = df_tahap.copy()
df_tahap_metrics.insert(0, 'bagian', 'training_loop')

# Gabungkan
metrics_final = pd.concat(
    [tabel_metrics, df_tahap_metrics],
    ignore_index=True,
    sort=False
)

# Simpan menjadi satu file
metrics_final.to_csv(
    f'M02_{NIM}_metrics.csv',
    index=False
)

print(metrics_final.to_string(index=False))
```

```
      bagian  run_id  seed parameter indeks      manual
autograd      numerik      rel_err      tahap  loss_akhir
gradient_check gradcheck      51      W1 (0, 0) -0.30343272 -
0.30343272 -0.30343272 1.00453007e-10      NaN      NaN
gradient_check gradcheck      51      W1 (0, 1)  0.15171636
0.15171636  0.15171636 2.95622632e-11      NaN      NaN
gradient_check gradcheck      51      W1 (1, 0)  0.15171636
0.15171636  0.15171636 2.95622632e-11      NaN      NaN
gradient_check gradcheck      51      W1 (1, 1) -0.07585818 -
0.07585818 -0.07585818 5.73360674e-11      NaN      NaN
gradient_check gradcheck      51      b1  (0,) -0.15171636 -
0.15171636 -0.15171636 2.95622632e-11      NaN      NaN
gradient_check gradcheck      51      b1  (1,)  0.07585818
0.07585818  0.07585818 5.73360674e-11      NaN      NaN
gradient_check gradcheck      51      W2  (0,) -0.11378727 -
```

```

0.11378727 -0.11378727 6.76755052e-11 NaN NaN
gradient_check gradcheck 51 W2 (1,) -0.07585818 -
0.07585818 -0.07585818 5.73360674e-11 NaN NaN
gradient_check gradcheck 51 b2 () -0.07585818 -
0.07585818 -0.07585818 5.73360674e-11 NaN NaN
training_loop NaN 51 NaN NaN NaN NaN
NaN NaN NaN train_rusak 0.69314718
training_loop NaN 51 NaN NaN NaN NaN
NaN NaN NaN train_benar 0.52281674

```

F. Tugas individu

Kerjakan ketiganya di sel-sel baru di bawah bagian ini.

1. **Perluasan jaringan.** Tambahkan neuron ketiga pada hidden layer: baris $[-1 \ 0.5]$ pada $W^{(1)}$, bias $0,25$, dan komponen $-0,5$ pada $W^{(2)}$. Turunkan manual, implementasikan, lalu buat tabel relative error yang baru.
2. **Batch dua contoh.** Tambahkan $x_2 = [-1 \ 3]$ dengan $y_2 = 0$, pakai rata-rata loss, dan jelaskan di langkah mana gradien kedua contoh dijumlahkan.
3. **Laporan diagnosis.** Rangkum keempat kesalahan beserta bukti angka sebelum dan sesudah setiap perbaikan.

1. Perluasan Jaringan

Neuron ketiga ditambahkan dengan:

$$W^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 1 \\ -1 & 0.5 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 0 \\ 0 \\ 0.25 \end{bmatrix}$$

dan:

$$W^{(2)} = [2 \ -1 \ -0.5]$$

Untuk $x = [2, -1]$:

$$z^{(1)} = W^{(1)}x + b^{(1)} = [1.5, 1, -2.25]$$

Setelah ReLU:

$$h = [1.5, 1, 0]$$

Logit:

$$z^{(2)} = W^{(2)}h + b^{(2)} = 2.5$$

Probabilitas:

$$p = \sigma(2.5) \approx 0.924142$$

Dengan $y=1$:

$$\delta^{(2)} = p - y \approx -0.0758582$$

Gradien layer output:

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} h = [-0.1137873, -0.0758582, 0]$$

$$\frac{\partial L}{\partial b^{(2)}} = -0.0758582$$

Gradien menuju hidden layer:

$$\frac{\partial L}{\partial h} = \delta^{(2)} W^{(2)} = [-0.1517164, 0.0758582, 0.0379291]$$

Karena neuron ketiga memiliki $z^{(1)} < 0$, turunan ReLU pada neuron tersebut adalah nol. Maka:

$$\delta^{(1)} = [-0.1517164, 0.0758582, 0]$$

Sehingga:

$$\frac{\partial L}{\partial W^{(1)}} = \begin{pmatrix} -0.3034327 & 0.1517164 \\ 0.1517164 & -0.0758582 \\ 0 & 0 \end{pmatrix}$$

dan:

$$\frac{\partial L}{\partial b^{(1)}} = [-0.1517164, 0.0758582, 0]$$

G. Pertanyaan analisis

1. Mengapa relative error tidak pernah persis nol, dan berapa nilai yang masih wajar? TODO
2. Apa yang terjadi pada tabel bila $\epsilon = 10^{-9}$? Jalankan dan jelaskan. TODO
3. Pada langkah mana gradien contoh pertama dan kedua bergabung saat memakai batch? TODO
4. Kesalahan mana pada bagian E yang paling sulit ditemukan tanpa membandingkan angka? TODO
5. Apa beda peran backpropagation dan optimizer? (maksimal tiga kalimat) TODO

Jawaban Analisis

1. Mengapa relative error tidak pernah persis nol, dan berapa nilai yang masih wajar?

Relative error biasanya tidak persis nol karena gradien numerik dihitung menggunakan pendekatan finite difference dan komputer memiliki keterbatasan presisi floating point. Pada praktikum ini, nilai relative error yang masih dianggap wajar adalah:

$$\text{relative error} < 10^{-5}$$

Semakin kecil nilainya, semakin dekat gradien numerik dengan gradien manual.

2. Apa yang terjadi pada tabel bila $\epsilon = 10^{-9}$?

Jika ϵ diperkecil menjadi:

$$\epsilon = 10^{-9}$$

relative error justru dapat menjadi lebih besar atau kurang stabil. Hal ini terjadi karena selisih

$$L(\theta + \epsilon) - L(\theta - \epsilon)$$

menjadi sangat kecil sehingga lebih mudah dipengaruhi oleh keterbatasan presisi floating point. Jadi, nilai ϵ yang terlalu kecil tidak selalu menghasilkan gradient checking yang lebih akurat.

3. Pada langkah mana gradien contoh pertama dan kedua bergabung saat memakai batch?

Gradien dari kedua contoh bergabung ketika gradien parameter dihitung dengan menjumlahkan kontribusi seluruh contoh dalam batch. Contohnya pada:

$$\frac{\partial L}{\partial W^{(2)}} = \sum_{i=1}^B \delta_i^{(2)} h_i$$

dan:

$$\frac{\partial L}{\partial W^{(1)}} = \sum_{i=1}^B \delta_i^{(1)} x_i^T$$

Pada kode, penggabungan tersebut terjadi pada operasi seperti `dz2_batch @ H_batch`, `dz1_batch.T @ X_batch`, dan `.sum()`.

4. Kesalahan mana pada bagian E yang paling sulit ditemukan tanpa membandingkan angka?

Kesalahan yang paling sulit ditemukan adalah memberikan `torch.sigmoid(logits)` ke `BCEWithLogitsLoss`. Program tetap dapat berjalan tanpa menghasilkan error, tetapi perhitungan loss dan gradien menjadi tidak sesuai karena `BCEWithLogitsLoss` sudah menerapkan sigmoid secara internal.

5. Apa beda peran backpropagation dan optimizer?

Backpropagation menghitung gradien loss terhadap setiap parameter model. Optimizer menggunakan gradien tersebut untuk memperbarui nilai parameter agar loss dapat berkurang. Jadi, backpropagation menghitung arah perubahan, sedangkan optimizer melakukan perubahan parameternya.

Checklist sebelum mengumpulkan

- [done] Identitas, seed, versi library, dan device tercantum.
- [done] Seluruh `TODO` dan `raise NotImplementedError` sudah diganti.
- [done] Turunan manual ditulis pada sel markdown bagian B.
- [done] Tabel relative error memuat sembilan baris dan seluruhnya lulus ambang.
- [done] Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- [done] Notebook lolos *Restart Kernel and Run All*.
- [done] Berkas: `M02_NIM.ipynb`, `M02_NIM.pdf`, `M02_NIM_metrics.csv`.